Makemodel Guide

`Makemodel` is the high-level entry point for authoring ModelFlow models. It takes a *markdown*-style text containing prose mixed with equations, expands LIST/DO templates, runs any tagged estimations, normalizes the result, and exposes a ready-to-solve `model` object.

This skill covers the authoring workflow end-to-end. For deeper detail on estimator backends (OLS, lmfit, EViews, constraints), see the **estimation** skill.

`Makemodel` lives in `modelconstruct_estimation.py` (and a non-estimation flavor in `modelconstruct.py` — use the `_estimation` one unless you specifically don't want estimation support).

TL;DR

```
from modelconstruct_estimation import Makemodel
from modelestimator_new import Estimate_nls

ls = Estimate_nls.with_defaults(input_df=df, smpl=(2002, 2018))

mm = Makemodel("""
A small consumption model.

> <estimator=ls> LOG(C) = C(1) + C(2)*LOG(Y) + C(3)*R
> <ident>       S = Y - C
> <ident>       I = S - DEF
""", input_df=df, estimator=ls)

mm.mmodel              # ready-to-solve modelflow model
mm.show                # print normalized FRMLs
mm.draw                # visualize the dependency graph
mm.estimation_report() # write makemodel_estimation_report.html
```

Equation input formats

`Makemodel` accepts two input formats — pick whichever is more natural for the situation. Both can be mixed inside the same text.

Format	When to use
> markdown	Fast authoring, Jupyter cells, code-style notation
LaTeX	Equations meant to be rendered in a paper or report; complex algebra; clear typographic separation

Format 1 — Markdown (> lines)

`Makemodel` reads markdown text where:

Line prefix	Meaning
`>`	An equation. No `FRML` keyword, no `$` terminator.
`>>`	Continues the previous equation (the two lines are joined with a space).
`>list ...`	A LIST or TLIST definition (see Templating section).
anything else	Prose. Ignored by the parser — you can mix documentation and equations freely.

Tags written inside `< ... >` attach metadata to the equation. `<estimator=...>` triggers estimation; `<ident>` declares an identity; others (`<smpl>`, `<caption>`, `<constraints>`, `<endo>`, `<stoc>`, etc.) modify behavior.

Continuation example

`A long equation broken over several lines:`

```
> <estimator=ls> DLOG(Y) = C(1) + C(2)*DLOG(X)
>>                        + C(3)*DLOG(Z)
>>                        + C(4)*(LOG(Y(-1)) - LOG(X(-1)))
```

is equivalent to

```
> <estimator=ls> DLOG(Y) = C(1) + C(2)*DLOG(X) + C(3)*DLOG(Z) + C(4)*(LOG(Y(-1)) - LOG(X(-1)
```

Format 2 — LaTeX (`\begin{equation}` blocks)

Equations can also be written as standard LaTeX `equation` environments. Two rules:

- **Every equation must have a `\label{eq:...}`.** Equations without a label are skipped by the parser — this is deliberate so you can put display-only math in the document.
- **Tags use the LaTeX comment marker.** Write `% @<tag>` on its own line inside the block, instead of the `<tag>` form used in `>` markdown.

```latex
\begin{equation}
\label{eq:consumption}
% @<estimator=ls>
\log(C_t) = C(1) + C(2) \cdot \log(Y_t) + C(3) \cdot R_t
\end{equation}
```

The parser translates the LaTeX body to ModelFlow notation through `latex_to_doable`. The most useful conversions:

LaTeX	Becomes	Notes
`_t`	(stripped)	Current period — the `t` index drops out
`_{t-1}`	`(-1)`	Lag operator
`_{t+1}`	`(+1)`	Lead operator
`^{x}`	`__{x}`	Templating dimension (drives `doable` expansion)
`^{x,y}`	`__{x}__{y}`	Multi-dimensional templating (up to 4 dims supported)
`\Delta X_t`	`diff(X)`	First difference
`\sum_{X}(expr)`	`sum(X, expr)`	Sum across a LIST named `X`
`\sum_{X=k}(expr)`	`sum(X k=1, expr)`	Sum starting at sublist position 1
`\max_{X}(expr)` / `\min_{X}(expr)`	`lmax(X, expr)` / `lmin(X, expr)`	List min/max
`\frac{a}{b}`	`(a)/(b)`	Fractions are flattened
`\sqrt{x}`	`sqrt(x)`	
`x^y`	`x**y`	Power (only when not used as a dim)
`\Phi(z)` / `\Phi^{-1}(z)`	`NORM.CDF(z)` / `NORM.PDF(z)`	Normal CDF / inverse
`\rho`, `\alpha`, `\beta`, `\tau`, `\sigma`, `\exp`	`rho`, `alpha`, `beta`, …	Common Greek letters
`\times`, `\cdot`	`*`	Multiplication
`\forall [agegroup=working]`	`[agegroup=working]`	Sublist filter (BLL `doable [...]`)
`\text{[banks=selected]}`	`[banks=selected]`	Same — alternative LaTeX wrapping
`\left`, `\right`, `\;`, `\,`, `\big`, `\nonumber`	(stripped)	Whitespace/sizing markup
`\begin{aligned}` … `\end{aligned}`, `\begin{split}` … `\end{split}`	(stripped)	Multi-line environments are flattened

If the body still contains a `\` after translation, `Makemodel` raises `ModelSpecificationError` telling you which fragment didn't translate. Add a regex or fix the LaTeX accordingly.

LIST definitions inside LaTeX LISTs can be written in inline math (`$ ... $`) so they render correctly in the document:

```
$List \; agegroup = \{16, 17, 18, 19, 20, 99, 100\}$

\begin{equation}
\label{eq:population_dynamics}
\forall [agegroup=middle] \; Population^{agegroup}_t =
    Population^{agegroup-1}_{t-1} - Dead^{agegroup-1}_{t-1}
\end{equation}
```

The parser harvests every `$List ... $` block in the text before processing equations, regardless of where they appear.

LaTeX with `<estimator=...>` The `% @<...>` tag accepts the same content as the `>`-markdown tag form, so estimation works identically:

```
\begin{equation}
\label{eq:phillips}
% @<estimator=nls_lmfit, constraints='C(2)>0'>
\Delta \log(P_t) = C(1) + C(2) \cdot UR_t + C(3) \cdot \Delta \log(P_{t-1})
\end{equation}
```

Mixing formats You can mix both formats in one `Makemodel` call. A typical use is LaTeX for the main estimated equation (so it renders nicely in a paper) and `>` markdown for accounting identities:

```
The behavioral equation:

\begin{equation}
\label{eq:consumption}
% @<estimator=ls>
\log(C_t) = C(1) + C(2) \log(Y_t)
\end{equation}

Identity definitions:

> S = Y - C
> I = S - DEF
```

Tags reference

Tag	Meaning
`<ident>`	Identity (definition). The default if no other tag is present.

Tag	Meaning
`<estimator=NAME>`	Run estimator `NAME` on this equation; bake the estimated coefficients in.
`<est=NAME>`	Short alias for `<estimator=NAME>`.
`<smpl=START END>`	Override the estimation sample for this equation.
`<caption='...'>`	Override the caption shown in the report.
`<constraints='...'>`	Constraints on estimated parameters (same syntax as inline `ST.`).
`<stoc>`	Stochastic equation — add an add-factor variable so the model can be calibrated to history.
`<exo>`	Exogenize the LHS — add a fixable add-factor variable.
`<fit>`	Generate a fitted-value variant alongside the equation.
`<endo=NAME>`	Override which variable on the equation is endogenous.
`<endo_lhs=FALSE>`	The LHS is not the endogenous variable (use with `<endo=...>`).
`<implicit>`	Equation is implicit (cannot be solved by simple substitution).

Multiple tags can appear in one `< ... >` block, separated by commas:

```
> <estimator=ls, smpl=2010 2018, constraints='C(2)>0'> DLOG(Y) = C(1) + C(2)*DLOG(X)
```

In a LaTeX equation, tags use the comment-marker form `% @<...>` on its own line inside the block — see the LaTeX section above.

Templating

LIST blocks — enumerated sublists

A `LIST` defines a named set with optional sublists. The pattern is `LIST name = sublist : values / sublist : values / ....` Inside equations, `{name}` expands to each value and `{sublist}` expands to the parallel value from another sublist.

```
>list banks = banks   : IB      SOREN  MARIE /
>>            country : DENMARK SWEDEN DENMARK /
>>            selected : 1       1      0

>list sectors = sectors : NFC SME HH
```

Loss equation, expanded for every bank × sector:

```
> doable LOSS__{banks}__{sectors} = HOLDING__{banks}__{sectors} * PD__{banks}__{sectors}
```

After expansion, `Makemodel` produces nine equations (3 banks × 3 sectors), e.g. `LOSS__IB__NFC = HOLDING__IB__NFC * PD__IB__NFC`.

TLIST — transposed list

When the natural reading is columns-by-rows rather than rows-by-columns, `TLIST` accepts the transposed form and rewrites it internally to a regular `LIST`:

```
>tlist banks = banks country selected /
>>             IB    DENMARK 1 /
>>             SOREN SWEDEN  1 /
>>             MARIE DENMARK 0
```

is equivalent to the `LIST` example above.

DO / ENDDO — explicit loops

For control over which list to loop and which conditions apply:

```
> do banks
>   £ comment: equations for bank {banks} in {country}
>   x_{banks} = 42
> enddo
```

The `£` symbol is the BLL comment character (comments are preserved into the FRML output but not parsed).

DOABLE — single-line DO

`doable` (also written `DOABLE`) is a one-line loop over every LIST appearing inside `{...}`:

```
> doable LOSS__{banks}__{sectors} = HOLDING__{banks}__{sectors} * PD__{banks}__{sectors}
```

Equivalent to nested `do banks / do sectors / ... / enddo / enddo` but more compact.

Filtering with `[sublist=value]`

You can restrict a `doable` to a subset using a sublist condition:

```
> doable [banks selected=1] x_{banks} = 42
```

Only generates equations for banks where `selected = 1`.

`replacements=` — Python-side string substitution

Apply one or more `(old, new)` string substitutions to the model text *before* parsing. Useful for two patterns:

Parsimony — write a placeholder once, substitute the real name at construction:

```
template = """
> <estimator=ls> LOG(CONS) = C(1) + C(2)*LOG(__INCOME)
"""

mm = Makemodel(
    template,
    replacements=[('__INCOME', 'GDP_REAL_DISP')],
    input_df=df,
    estimator=ls,
)
```

Same template, many entities — write the model once, then loop:

```
template = """
> <estimator=ls> LOG(LOANS__dim) = C(1) + C(2)*LOG(GDP__dim)
> <ident>       NPL__dim = LOANS__dim * NPL_RATIO__dim
"""

bank_models = {
    bank: Makemodel(
        template,
        replacements=[('__dim', f'_{bank}')],
        input_df=df,
        estimator=ls,
    )
    for bank in ['IB', 'SOREN', 'MARIE']
}
```

Accepted shapes:

```
replacements=('OLD', 'NEW')              # single pair
replacements=[('A', 'B'), ('C', 'D')]    # list, applied in order
replacements=[]                          # no-op (default)
```

Notes: - Plain `str.replace` — no regex. Make sentinels distinctive (`__dim`, `__INCOME`, …) so they don't collide. - Substitution is order-sensitive: later pairs see earlier output. - Operates on the entire model text — including prose between `>` lines. - Use `LIST` for "all banks in one model"; use `replacements=` for "one model per bank."

Estimation integration

Any equation tagged with `<estimator=NAME>` is estimated during `Makemodel.__post_init__`. The estimated coefficients are baked into the equation before normalization, so the resulting `mmodel` has numeric coefficients in place of `C(1)`, `C(2)`, …

Basic estimation

```
from modelconstruct_estimation import Makemodel
from modelestimator_new import Estimate_nls

# Factory captures shared defaults
ls = Estimate_nls.with_defaults(input_df=df, smpl=(2002, 2018))

mm = Makemodel("""
> <estimator=ls> DLOG(Y) = C(1) + C(2)*DLOG(X) + C(3)*(LOG(Y(-1)) - LOG(X(-1)))
> <ident>       Z = Y * 0.5
""", input_df=df, estimator=ls)
```

The estimator name `ls` is resolved from the caller's namespace (so `Estimate_nls.with_defaults(...)` stored in a local variable just works), or from an explicit `estimator_classes={'ls': ...}` mapping if you prefer.

Estimation-related `Makemodel` kwargs

Kwarg	Purpose
`input_df=`	DataFrame fed to each estimator. Each estimator may override via its own `input_df`.
`estimator=`	Default estimator for `<estimator>` flags with no value.
`smpl=`	Default estimation sample. `<smpl=START END>` per equation overrides.
`estimator_kwargs={}`	Shared kwargs (e.g. `caption`, `add_add_factor`) passed to every estimator.
`estimator_classes={}`	Explicit name → class mapping (alternative to caller-namespace resolution).
`estimator_namespace={}`	Namespace dict for resolving `<estimator=ls>`-style names.

Constraints on estimated parameters

Two equivalent forms:

Inline `ST.` clause (constraints stay with the equation):

```
> <estimator=ls> Y = C(1) + C(2)*X ST. C(1)~0.5; C(2)>0; C(2)<=1
```

`<constraints=...>` tag (separate from the equation):

```
> <estimator=ls, constraints='C(1)~0.5; C(2)>0; C(2)<=1'> Y = C(1) + C(2)*X
```

Recognized constraint forms (full reference in the **estimation** skill):

Form	Meaning
`NAME=[lo, hi]`	Bounded between `lo` and `hi`
`NAME > x` / `>= x`	Lower bound
`NAME < x` / `<= x`	Upper bound
`NAME = value`	Fixed (not estimated)
`NAME ~ value`	Initial value (free to change)
`NAME := expr`	Derived from other params

Constraints work with `Estimate_nls_lmfit`. OLS and EViews raise an error if constraints are supplied.

Inspecting estimations

After construction:

```
mm.estimation_records       # list of dicts, one per estimated equation
mm.estimation_report()      # writes HTML report (all estimations)
mm.markdown_with_estimation  # original markdown + inline coefficient tables
mm.render_est               # interactive display of the above
```

Each `estimation_records` entry contains:

```
{
    'equation_index': int,           # position in the original model
    'frmlname': str,                 # original FRML name flag
    'estimator': str,                # display name of the estimator
    'smpl': tuple,                   # sample used
    'original_expression': str,      # before baking
    'baked_expression': str,         # with coefficients substituted
    'estimator_object': EstimatorBackend,  # the live estimator
}
```

Use `record['estimator_object'].mfresult.get_html_report()` for the full single-equation HTML view.

Outputs and useful attributes

Attribute	What it is
`mm.mmodel`	The full ready-to-solve `model` (cached). Use this for forecasting/simulation.
`mm.clean_model`	A `model` containing only the core equations (no add-factor or fitted variants).
`mm.add_model`	A `model` that *calculates* add factors from historic data.
`mm.normal_frml`	The fully normalized FRML text.
`mm.clean_frml`	Normalized FRMLs with `ADD`/`EXO`/`FIT` options stripped.
`mm.show`	Prints `normal_frml`.
`mm.draw`	Plots the dependency graph (uses `modelnet`).
`mm.modellist`	Dict of parsed LIST definitions.
`mm.showlists`	Pretty-prints the LIST definitions.
`mm.render`	Renders the markdown source (equations + prose) in a notebook.
`mm.render_est`	Same as `render`, plus estimation tables inline.
`mm.estimation_records`	List of estimation result records (see above).

Aligning to historic data — `init_addfactors`

For models with `<stoc>` or `<exo>` tags, ModelFlow generates add-factor variables (`*_A`). `init_addfactors` computes the add factors needed to make the model exactly reproduce a historic dataframe:

```
aligned_df = mm.init_addfactors(df, start=2002, end=2020, show=True, check=True)
# aligned_df has the add-factor columns filled in
# show=True prints them; check=True re-simulates to verify
```

After this, `mm.mmodel(aligned_df, ...)` reproduces history exactly, and any change to a variable produces a counter-factual.

Combining models — `+`

Two `Makemodel` instances can be concatenated:

```
core   = Makemodel(core_equations)
shocks = Makemodel(shock_equations)
full   = core + shocks
full.mmodel   # contains both blocks
```

LIST definitions are merged. Useful for keeping a base model in one cell and scenario overrides in another.

Authoring from a Jupyter cell — the %%Makemymodel magic

`%%Makemymodel` is the notebook-cell front end to `Makemodel`. Instead of passing model text to `Makemodel("""...""")` in Python, you write the markdown/LaTeX model **as the body of a cell**, put options on the magic line, and the magic builds a `Makemodel`, stores it in the notebook namespace under the name you gave, and renders it. It lives in `modeljupytermagic.py` and delegates all the real work to `Makemodel` / `display_model` from `modelconstruct_estimation.py` — so **every equation format, tag, template, and estimation feature above applies unchanged**. This section only covers the magic layer.

Loading

The magics register via decorators that run **at import time** — there is no `%load_ext`. Import once per kernel:

```
import modeljupytermagic          # registers %%Makemymodel, %latexflow, %%graphviz, ...
```

If import prints `no magic`, IPython wasn't importable when the module loaded (the whole block is wrapped in `try/except`); re-import inside a live IPython kernel.

Example

```
import modeljupytermagic
from modelestimator_new import Estimate_nls
ls = Estimate_nls.with_defaults(input_df=df, smpl=(2002, 2018))
```

```
%%Makemymodel con input_df=df estimator=ls smpl="(2002, 2018)"
A small consumption model.

> <estimator=ls> LOG(C) = C(1) + C(2)*LOG(Y) + C(3)*R
> <ident>        S = Y - C
> <ident>        I = S - DEF
```

After the cell runs, `con` is a `Makemodel` in the namespace (name = first token on the line); the cell renders the markdown with inline estimation tables. `con.mmodel`, `con.show`, `con.draw`, `con.estimation_report()` all work as above. The magic **discards** the returned object (`_ = _mdmodel_impl(...)`) — grab the model **by name**, not from the cell output.

Line vs cell form

Form	Use
`%%Makemymodel name [opts]` + cell body	Build (or extend) a model from the cell's markdown/LaTeX.
`%Makemymodel name [opts]`	**Re-render / rebuild an existing model** with no new content (cell is `None`) — e.g. to re-display a segmented model once all pieces are in, or re-render with different switches.

Both are backed by the same `_mdmodel_impl(line, cell)`.

The magic line — name and options

`get_options()` tokenizes the line with `shlex` (POSIX):

- **First token = the model name**, and the resulting `Makemodel` is stored under exactly that name. Empty line → name defaults to `test`.
- **Remaining tokens = options**, each `key=value` or a bare flag:
  - Bare flag (`show`, `draw`, `latex`, `segment`) → `True`.
  - `key=0` / `key=False` → `False` (this is how you turn a default-on switch **off**, e.g. `render_est=0`). Bare flags only turn things *on*.
  - It's `shlex` POSIX, so **quote values containing spaces**: `smpl="(2002, 2018)"`, `caption="My model"`. Only the first `=` splits key from value.

Option values that name Python objects are resolved by `_resolve_option`: first `ast.literal_eval(value)` (so `replacements="[('__dim','_IB')]"` and `smpl="(2010, 2019)"` work as literals), then a lookup **by name in the notebook namespace** (so `input_df=df` and `estimator=ls` resolve to live objects). If a value is neither a literal nor a known name, a warning is printed (`Warning: input_df=dff not found in namespace and not a literal.`) and the option **silently falls back to its default** — watch for that warning.

Options mapped onto `Makemodel(...)` kwargs:

Option	Effect
`input_df=NAME`	DataFrame passed to estimators for tagged equations.
`estimator=NAME` / `est=NAME`	Default estimator for `<estimator>` flags (`estimator` wins; `est` is the fallback alias).
`smpl="(START, END)"`	Default estimation sample; per-equation `<smpl=…>` still overrides.

Option	Effect
`replacements="[('OLD','NEW'), …]"`	String substitutions applied before parsing.
`funks=NAME`	List of user functions available inside the model.
`spec=markdown` | `spec=latex`	Which renderer `display_model` uses (default `markdown`).

The magic always passes `estimator_namespace=ip.user_ns`, so `<estimator=ls>` resolves against the notebook namespace with no extra wiring.

Rendering / output switches (default on except where noted):

Option	Default	Effect
`render_est`	on	Render `markdown_with_estimation` (source + inline coefficient tables).
`render`	on	Used only when `render_est=0`: render plain model text (no estimation tables).
`render_list`	on	Include LIST definitions in the render. See caveat below before setting `render_list=0`.
`show`	off	Also `print` the normalized FRMLs (`emodel.show`).
`draw`	off	Also draw the dependency graph (suppressed when `display` is on).
`latex`	off	Build a LaTeX version and open a PDF via `LatexRepo(...).pdf(pdfopen=True)`; prints `no latex` on failure.
`display`	off	Verbose: render the model text, list the segments it was built from, and `print(emodel)`.

Caveat — `render_list=0`: with `render_est` on (the default), passing `render_list=0` renders `emodel.markdown_with_estimation_no_list`, which is **not defined on `Makemodel`** and raises `AttributeError`. Only turn off `render_list` together with `render_est=0` (which renders the plain no-list text and is safe), or leave `render_list` on.

Segmented models — one model from several cells

Spread a long model across cells, giving each a segment name with `segment=<segname>`. The magic keeps `NAME_dict` in the namespace mapping segment → cell text, and reassembles the whole model on each run:

```
%%Makemymodel npl segment=lists
>list banks = banks : IB SOREN MARIE
```

```
%%Makemymodel npl segment=behavior input_df=df estimator=ls
> <estimator=ls> LOG(LOSS__{banks}) = C(1) + C(2)*LOG(GDP)
```

```
%%Makemymodel npl segment=identities
> doable NPL__{banks} = LOSS__{banks} * FACTOR__{banks}
```

- Each `segment=SEG` run stores/overwrites `NAME_dict[SEG]` with the cell.
- **Segments whose name starts with `list` or `text` are display-only** for that cell: they render and return, but their content is still stored and folded into the assembled model (so you can show list tables / prose in place).
- When building, all `list…` segments plus the current cell are concatenated, so list definitions are always in scope for the equations.
- Re-run a single segment cell to edit just that part; the model rebuilds from the full `NAME_dict`. `%Makemymodel npl` (line form) rebuilds/re-renders from the accumulated segments — use it after the last piece, or with `display`/`latex`.

Naming segments `lists`, `behavior`, `identities` is convention; only the `list`/`text` *prefixes* are special.

Sibling magics

`modeljupytermagic.py` registers other cell magics that share the same `get_options()` line parser (first token = name):

Magic	Purpose
`%%latexflow name`	Build from a LaTeX document via `model_latex.latextotxt` → `model.from_eq` (older path; prefer `Makemymodel`).

Magic	Purpose
`%%latexmodelgrab name`	Build via the dataclass `a_latex_model` (`model_latex_class`); supports `segment=`, `all`, `render`, `display`.
`%%graphviz name`	Render a Graphviz graph through `model.display_graph`.
`%%dataframe name`	Turn a whitespace/tab table in the cell into a DataFrame (options `t`, `melt`, `periods=`, `prefix=`, `start=`, `show`); pushes `NAME` / `NAME_melted`. Yearly data.
`%%modeleviews name`	Run EViews commands from the cell (requires `pyeviews`; developmental).

Recipes

Quick identity-only model

```
mm = Makemodel("""
> Y = C + I + G + X - M
> S = Y - T
> DEF = G - T
""")
mm.mmodel
```

(No `<ident>` needed — equations without tags default to identity.)

One estimated equation in a larger model

```
mm = Makemodel("""
Consumption is estimated; the other equations are identities.

> <estimator=ls> LOG(C) = C(1) + C(2)*LOG(Y) + C(3)*R
> S = Y - C
> I = S - DEF
""", input_df=df, estimator=ls)
```

Mixed estimators

```
mm = Makemodel("""
> <estimator=nls_lmfit> DLOG(C) = C(1) + C(2)*DLOG(Y) + C(3)*(LOG(C(-1)) - LOG(Y(-1))) ST. C
> <estimator=ols>       LOG(I) = C(1) + C(2)*LOG(Y) + C(3)*R
> <ident>               S = Y - C
""", input_df=df)
```

Stochastic equation (add-factor variant)

```
mm = Makemodel("""
> <estimator=ls, stoc> LOG(C) = C(1) + C(2)*LOG(Y)
""", input_df=df, estimator=ls)

mm.init_addfactors(df_history)  # populates LOG_C_A so model reproduces history
```

Bank-by-bank credit-loss model

```
template = """
> <estimator=ls> LOG(LOSS__bank) = C(1) + C(2)*LOG(GDP__bank) + C(3)*UR__bank
> <ident>       NPL__bank = LOSS__bank * NPL_FACTOR__bank
"""

models = {
    b: Makemodel(template, replacements=[('__bank', f'_{b}')], input_df=df, estimator=ls)
    for b in ['IB', 'SOREN', 'MARIE']
}
```

Many sectors inside a single model

```
mm = Makemodel("""
>list sectors = sectors : NFC SME HH

> doable <estimator=ls, stoc> LOG(LOSS__{sectors}) = C(1) + C(2)*LOG(GDP)
""", input_df=df, estimator=ls)
# Three estimated equations: LOSS_NFC, LOSS_SME, LOSS_HH
```

LaTeX-authored model

```
mm = Makemodel(r"""
A small Phillips-curve model.

$List \; lags = \{1, 2, 3, 4\}$

\begin{equation}
\label{eq:phillips}
% @<estimator=nls_lmfit>
\Delta \log(P_t) = C(1) + C(2) \cdot UR_t + C(3) \cdot \Delta \log(P_{t-1})
\end{equation}

\begin{equation}
\label{eq:unemployment}
% @<ident>
UR_t = 100 \cdot (1 - E_t / L_t)
```

```
\end{equation}
""", input_df=df)
```

Same equation, two notations

These two `Makemodel` calls produce identical models:

```
mm_md = Makemodel("""
> <estimator=ls> LOG(C) = C(1) + C(2)*LOG(Y) + C(3)*LOG(C(-1))
""", input_df=df, estimator=ls)

mm_tex = Makemodel(r"""
\begin{equation}
\label{eq:consumption}
% @<estimator=ls>
\log(C_t) = C(1) + C(2) \log(Y_t) + C(3) \log(C_{t-1})
\end{equation}
""", input_df=df, estimator=ls)
```

Common pitfalls

- **`<estimator=ls>` not found** — make sure `ls` is in the caller's scope, or pass `estimator_classes={'ls': ls}` / `estimator_namespace=locals()`.
- **Equations don't expand inside DO blocks** — check that you're using `{name}` (with braces) not `name`.
- **`replacements=` matched too much** — your sentinel collided with a real token. Use distinctive prefixes (`__bank`, not `BANK`).
- **`replacements=` is plural** — the kwarg name is `replacements`, not `replace` (a common typo).
- **Estimator can't take constraints** — OLS and EViews backends raise on any `ST.` clause or `<constraints=...>` tag. Switch to `Estimate_nls_lmfit`.
- **Estimation sample shrinks unexpectedly** — OLS drops NaN/inf rows from lags and `LOG()` transforms; the actual sample is reported in `estimator_obj.estimation_smpl`.
- **`mm.mmodel` is cached** — re-running estimation requires constructing a new `Makemodel`. The cached model property doesn't refresh automatically.
- **LaTeX equation skipped silently** — every equation environment needs `\label{eq:...}`. Equations without a label are treated as display-only and dropped by the parser. Add a label even if you don't reference it elsewhere.
- **`ModelSpecificationError: Some LaTeX has survived`** — a LaTeX construct didn't translate (typically a Greek letter or operator not in the conversion table). Either rewrite that part in plain notation, or extend the translation tables in `latex_to_doable` (in `modelconstruct_estimation.py`).

- **Raw strings for LaTeX** — pass LaTeX-containing model text as `r"""..."""` so backslashes survive intact: `\Delta` would otherwise be interpreted by Python.

`%%Makemymodel` magic pitfalls

- **Model not found after the cell** — the object is stored under the *first token* on the magic line, not returned. `%%Makemymodel con …` → use `con`, not the cell output.
- **`input_df=df` "not found" warning** — option values are resolved from the notebook namespace; define `df`/`ls` in an earlier cell and check spelling. A bad name warns and falls back to the default (often `None`), which then trips `input_df is required when any equation has an <estimator=…>`.
- **Spaces in an option value** — quote it: `smpl="(2002, 2018)"`, not `smpl=(2002, 2018)` (the space would start a new token).
- **Turning a switch off** — use `=0` / `=False` (`render_est=0`), not just omitting a bare flag; bare flags only turn things *on*.
- **`render_list=0` with estimation on** — raises `AttributeError` (`markdown_with_estimation_no_list` doesn't exist on `Makemodel`). Pair it with `render_est=0`, or leave `render_list` on. (Latent bug in `_mdmodel_impl`.)
- **Magics missing / `no magic` printed** — import `modeljupytermagic` inside a running IPython kernel; the decorators register only when IPython is importable.
- **Editing one segment** — re-running a `segment=` cell overwrites just that entry in `NAME_dict`; the model reassembles from all segments, so you don't need to re-run the others.

Related modules and skills

- **estimation** skill — deeper detail on the estimator backends (OLS / lmfit / EViews), constraint forms, initial-value strategies, and authoring new backends.
- `modelconstruct.py` — same dataclass without estimation hooks. Use the `_estimation` variant unless you specifically want the smaller surface.
- `modelclass.py` — the underlying `model` class that `mm.mmodel` returns.
- `modelnormalize.py` — handles `DLOG`/`DIFF`/`MOVAVG` rewriting; called from `Makemodel.__post_init__`.
- `modelmanipulation.py` — `tofrml`, `dounloop`, `sumunroll`; the LIST/DO expansion engine.
- `modelpattern.py` — `kw_frml_name` extracts tag values like `<estimator=NAME>`.